

JACUSA2 manual

Michael Piechotta
michael.piechotta@gmail.com

March 6, 2026

Contents

1	Introduction	3
1.1	Variant calling	4
1.2	Reverse transcriptase arrest events	4
2	Download	5
2.1	Installation and requirements	5
2.2	Migrating from JACUSA1 to JACUSA2	5
2.3	<i>In silico</i> and example data	5
2.3.1	Variant calling	5
2.3.2	Reverse transcriptase arrest events	6
3	Input/Output	6
3.1	Processing BAM files	7
3.1.1	Remove duplicates for variant calling	7
3.1.2	Library type and strand information	7
3.2	BED-like search region	8
3.3	General output format	8
4	Artefact/feature filter	9
5	Usage	11
5.1	Variant detection	11
5.1.1	call-1	11
5.1.2	call-2	12
5.1.3	call-* output format	12
5.2	<i>pileup</i>	12
5.2.1	<i>pileup</i> output format	12
5.3	*-arrest - Reverse transcriptase arrest events	12
5.3.1	rt-arrest - 2 conditions	12
5.3.2	lrt-arrest - 2 conditions	13
6	Optional data	13
6.1	Adding INDEL statistics	14
6.2	Deletion output format	14
6.3	Adding read/base stratification	14
7	Description of command line options	14
7.1	Input / Output	14
7.1.1	Input BAM files	14
7.1.2	Output file	14

7.1.3	Output artefacts to separate file	15
7.1.4	Show all sites	15
7.2	Input BED file	15
7.3	Reference fasta file	15
7.4	Library type	15
7.5	Read/base stratification	15
7.6	Show INDEL statistics	16
7.6.1	Show insertion statistics	16
7.6.2	Show deletion statistics	16
7.7	General filtering	16
7.7.1	Filter by mapping quality	16
7.7.2	Filter by base call quality	16
7.7.3	Enforce/override base call quality	16
7.7.4	Filter by minimal coverage	17
7.7.5	Limit maximal depth	17
7.7.6	Filter by flag(s)	17
7.7.7	Retain by flag(s)	17
7.7.8	Filter by number of hits	18
7.7.9	Filter by number of mismatches	18
7.8	Artefact/feature filter	18
7.9	Thread related	18
7.9.1	Number of parallel threads	18
7.9.2	Actual thread window size	19
7.9.3	Reserved thread window size	19
7.10	Test-statistic options	19
7.11	Misc	19

8 Used libraries 19

1 Introduction

JAVA framework for accurate Variant assessment (JACUSA2) is a one-stop solution to detect single nucleotide variants (SNVs) and reverse transcriptase induced arrest events in Next-generation sequencing (NGS) data.

JACUSA2 is a direct successor of JACUSA1 — JACUSA1 is hereby deprecated and won't be continued. All methods from JACUSA1 (*call-1*, *call-2*, and *pileup*) are available in JACUSA2.

The new release of JACUSA2 offers great performance enhancements ($\approx 3\times$ faster) for existing methods and adds new methods (*rt-arrest* and *lrt-arrest*) to identify read arrest events by means of comparing read through and read arrest counts. *lrt-arrest* is a combination of *call-2* and *rt-arrest* that allows to identify linked variants and arrest events.

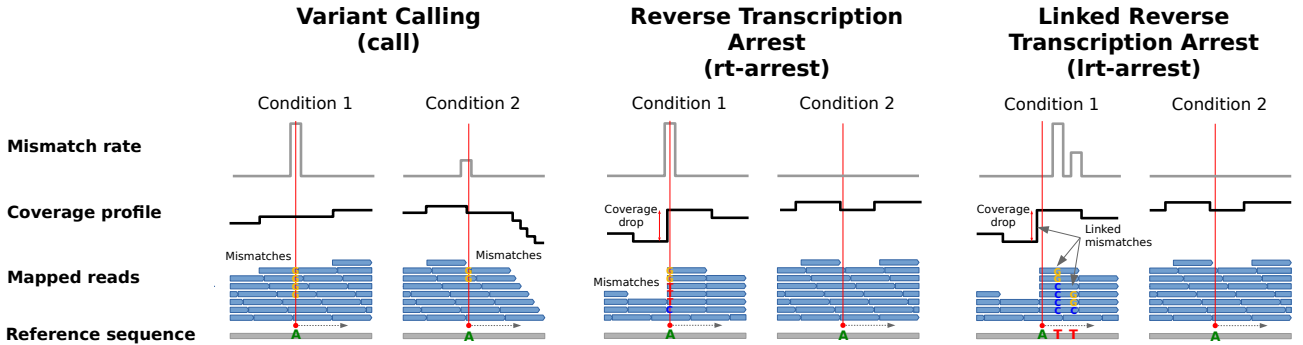


Figure 1: Schematic summary of JACUSA2 methods and underlying data.

Robust identification of variants has proven to be a daunting task due to artefacts specific for NGS-data and employed mapping strategies. We implement various artefact/feature filters that reduce the number of false positives (see section 4). A new feature filter has been added to JACUSA2 to filter candidate variants or arrest events based on an external file. Some artefact filters have been removed from JACUSA1 in favour of the rewritten R helper package called JACUSA2helper¹.

Another new feature, are optional data that can be added to the output of some methods. Such optional data are INDEL counts and associated differential statistics and tagging reads with base substitutions. A user can provide base substitution(s) to partition reads into two sets (see Section 6.3).

In JACUSA, a site consists of contig, position (start and stop), and strand information. The 1:1 relation of a site to a line in the output of JACUSA1 had to be extended to make more complex data structures possible while maintaining a clear file format. JACUSA2 allows data associated to a site to be stratified by additional variables and spread along multiple lines. For instance, read tagging can be used to stratify reads based on a base substitution. In the output, there will be two lines for each site. The first line will contain base count information for all reads while the second line will contain only base counts from reads with the chosen base substitution.

The combined variant calling and arrest event discovery *lrt-arrest* offers information regarding the arrest position of reads that overlap with a site. The number of lines per site depends on overlapping reads and their arrest positions. The arrest position of a read depends on the employed library type and can unambiguously be determined only for stranded library types. Each unique arrest position will result in a new line in the output.

JACUSA2 employs a window-based approach to traverse BAM [Li et al., 2009] files, featuring highly parallel processing and utilizing the htsjdk² framework.

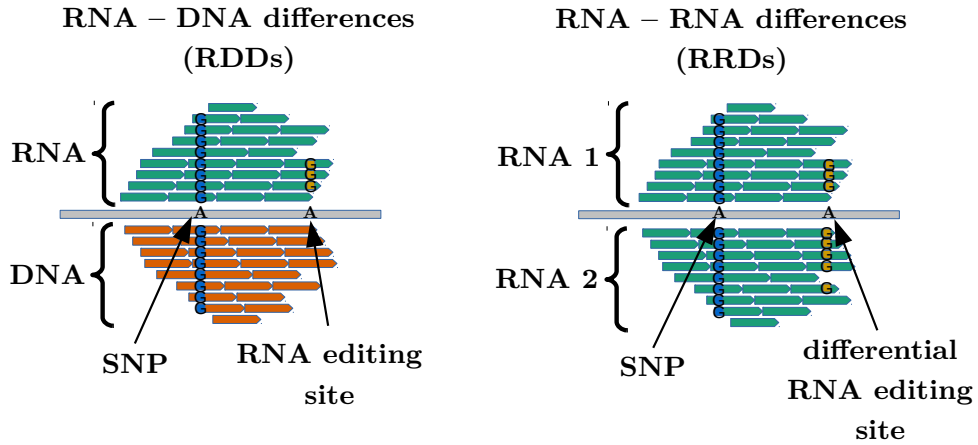


Figure 2: Two procedures to identify RNA-editing sites in JACUSA2. Comparing DNA vs. RNA to elucidate RNA-DNA-differences (RDDs) and comparison of RNA sequencing samples to find RNA-RNA-differences (RRD).

1.1 Variant calling

JACUSA2 has been extensively evaluated and optimized to identify RNA editing sites in RNA-DNA and RNA-RNA sequencing samples (see Figure 2). Checkout the original publication and the supplementary material of JACUSA1 [Piechotta et al., 2017] if you are interested in details regarding the test-statistic.

1.2 Reverse transcriptase arrest events

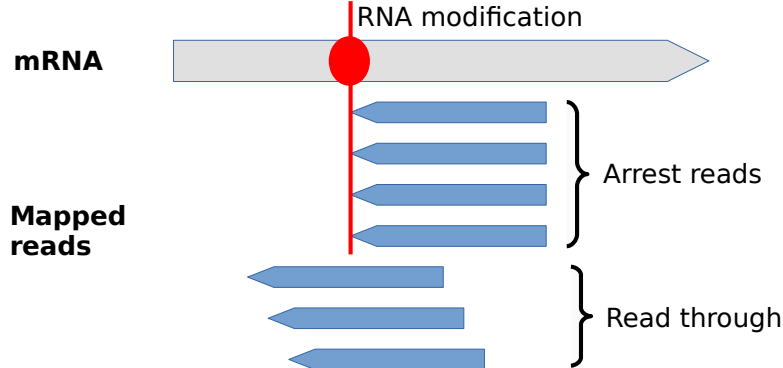


Figure 3: Schematic depiction of arrest events that have been induced by modifications that influence reverse transcription during library preparation.

Reverse transcriptase arrest events can be induced during library preparation (see Figure 3). They are identified by reads that exhibit shorter than expected read length due to premature termination during first strand synthesis. For each site, a vector of read through and read arrest counts is constructed and modelled with a Beta-Binomial distribution. We estimate the parameters of the distribution with the method presented by Minka³.

¹Check: <https://github.com/dieterich-lab/JACUSA2helper>

²Check: <https://github.com/samtools/htsjdk>

³Check: <https://tminka.github.io/papers/dirichlet/minka-dirichlet.pdf>

2 Download

The latest version of JACUSA2 can be obtained from <https://github.com/dieterich-lab/JACUSA2/releases>. Also check <https://github.com/dieterich-lab/JACUSA2helper> for updates.

2.1 Installation and requirements

JACUSA2 does not need any configuration but requires a correctly configured Java environment. We developed and tested JACUSA2 with Java 17. If you encounter Java-related problems, consider switching to Java 17.

2.2 Migrating from JACUSA1 to JACUSA2

There are several important changes to the command line interface:

- ALL two dash options “-option [...]” from JACUSA1 have been removed. ONLY single dash “-” options, e.g.: “-c 10” are supported and NOT “-coverage 10”.
- Use “-filterNH” and “-filterNM” instead of “-filterNH” and “-filterNM”.
- CLI option to provide library type has changed: JACUSA1: “-P Lib1,Lib2” → JACUSA2: “-P1 Lib1 -P2 Lib2”. See Section 3.1.2.
- Artefact/feature filters have now named options, e.g.: JACUSA1: “-a H:1” → JACUSA2: “-a H:condition=1”. See Section 7.8 for details.
- Test-Statistic options “-u <STAT>” have named options.
- The output of *call-1* does not include the simulated condition.
- In JACUSA2 the base of the distance measure for artefact filters has changed, e.g.: “-B:distance=*d*”. A distance $d = 1$ in JACUSA1 meant that the features were immediately adjacent. In JACUSA2, $d = 0$ corresponds to adjacent features. Adjust your JACUSA2 calls according to the following relation: $d_{JACUSA1} = d_{JACUSA2} + 1$.

A “##” prefixed header line has been added to the default output format of JACUSA2. The header line contains version and command line info. There is also a new version of JACUSA2helper⁴ to support downstream analysis of JACUSA2 output. The old version of JACUSAhelper has been declared deprecated and won’t be maintained anymore.

2.3 *In silico* and example data

2.3.1 Variant calling

You can choose between different *in silico* setups to detect variants.

The gDNA vs. cDNA represents the typical data setup that is encountered in detection of RNA editing sites via comparing genomic and transcriptomic sequencing reads. In this setup, variants have been only imputed to the cDNA BAM file.

The cDNA vs. cDNA data setup can be interpreted as representing allele specific expression of single variants or differential RNA editing. In this setup, variants with pairwise different base frequencies have been imputed into both cDNA BAM files. Additionally, to make the identification of variants more challenging, SNPs with pairwise similar base frequencies have been included to both BAM files. SNPs should not be identified as true positive sites.

gDNA data has been simulated with art⁵ and cDNA reads have been simulated with flux simulator⁶. Read simulations have been restricted to the corresponding first chromosome of human. Simulated BAM files have

⁴Check: <https://github.com/dieterich-lab/JACUSA2helper/>

⁵Check: <http://www.niehs.nih.gov/research/resources/software/biostatistics/art>

⁶Check: <http://sammet.net/confluence/display/SIM/Home>

Table 1: Detailed description of available *in silico* data for variant calling. Data has been simulated on human chromosome 1 using hg19.

Setup	File	Description
gDNA vs. cDNA ⁷	gDNA.bam	Simulated genomic DNA
	cDNA.bam	Simulated RNA-Seq data
	variants.txt	Coordinates of imputed variants and their target and sampled frequencies.
gDNA vs. cDNA ⁸	cDNA_1.bam	Simulated RNA-Seq data for 1st condition.
	cDNA_2.bam	Simulated RNA-Seq data for 2nd condition.
	variants.txt	Coordinates of imputed variants and their target and sampled frequencies.
	snps.txt	Coordinates of imputed SNPs. In both BAM files matching SNPs have the same target frequency but different effective or sampled frequencies. The shape parameter determines how much the sampled frequency will deviate from the target frequency. The suffixes: 1 and 2 correspond to the respective BAM file.

been processed and only reads with mapping quality ≥ 20 have been retained. Check Table 1 for details about contents of available data check links in the footnote.

2.3.2 Reverse transcriptase arrest events

We have downloaded primary sequencing data from [Zhou et al., 2018]. From the resulting read sets, we retained only uniquely mapping reads to 18S and 28S rRNAs with mapping quality ≥ 20 . [Zhou et al., 2018] map RNA modification of pseudouridine (Ψ) by chemically modifying pseudouridines with carbodiimide (+CMC) and detecting arrest events that are induced by reverse transcription stops in high-throughput sequencing under 3 different conditions: HIVRT, SIIIRTMn, and SIIIRTMg.

The archive https://data.dieterichlab.org/s/arrest_events consists of:

***.bam, *.bam.bai** BAM files and BAM indices for different 3 conditions HIVRT, SIIIRTMg, and SIIIRTMn. Each condition is a pairwise comparison of CMC(+CMC) and mock-treated(-CMC) samples (6 BAM + 6 index files).

fournier_db.txt Parsed and coordinate adjusted list of known modifications for human 18S and 28S rRNAs according to Fournier lab’s 3D rRNA modification database [Piekna-Przybylska et al., 2007].

README.txt Summary of referenced data and original data sources. Read for details.

3 Input/Output

All JACUSA2 methods require sorted and indexed BAM⁹ files. SAM/BAM is a standardized file format for efficient storage of alignments. Furthermore, JACUSA2 requires that the reference sequence is available either through the “MD”-tag ¹⁰ in BAM files or by providing the reference sequence in indexed FASTA format with the command line option “-R <reference.fasta>”. Make sure that “<reference.fasta>” is the reference sequence that has been used for mapping. The “MD” field contains mismatch information that allows to perform variant calling without providing the reference sequence.

Check the manual of SAMtools/BCFtools¹¹ or picard tools¹² for how to use the respective tool to convert your alignment files to valid JACUSA2 input BAM.

⁷Check: gDNA vs. cDNA: https://data.dieterichlab.org/s/gDNA_VS_cDNA

⁸Check: cDNA vs. cDNA: https://data.dieterichlab.org/s/cDNA_VS_cDNA

⁹Check: <https://samtools.github.io/hts-specs/SAMv1.pdf>

¹⁰Check: <https://samtools.github.io/hts-specs/SAMtags.pdf>

¹¹Check: <https://www.htslib.org/>

¹²Check: <http://broadinstitute.github.io/picard/>

3.1 Processing BAM files

In the following, commands for SAMtools¹³ are presented.

To sort and index your raw BAM files, perform the following sequence of commands:

```
SAM → BAM samtools view -Sb mapping.sam > mapping.bam
```

```
sort BAM samtools sort -o mapping.sorted.bam mapping.bam
```

```
index BAM samtools index mapping.sorted.bam
```

Check if your BAM file contains “MD”-tag, if you want to provide reference sequence information via this tag. When your BAM files do not have the “MD” tag properly set, use SAMtools:

```
samtools calmd mapping.sorted.bam reference.fasta > \  
mapping.sorted.MD.bam
```

“<reference.fasta>” is required to be identical to the reference that was used for mapping.

3.1.1 Remove duplicates for variant calling

It is a recommended pre-processing step to remove duplicated reads when identifying variants - **omit this step for *rt-arrest* and *lrt-arrest***. Reads that are terminated prematurely during library preparation will falsely be identified as PCR duplicates and removed from the final output. This will dramatically reduce sensitivity for arrest event identification.

Duplicated reads occur mostly due to PCR-artefacts. They are likely to harbour false variants and most statistical tests require independently sampled reads. In the following, commands for picard tools are presented:

```
java -jar MarkDuplicates.jar \  
I=mapping.sorted.bam O=dedup_mapping.sorted.bam \  
M=duplication.info
```

Most tools either remove duplicated reads from the final output or assign the value 1024 to the flag¹⁴ field of those reads. In the later case, invoke JACUSA2 with the additional command line option “-F 1024” to filter reads that have been marked as duplicates (see Section 7.7.6).

3.1.2 Library type and strand information

JACUSA2 supports stranded paired end (PE) and single ends (SE) reads. **Warning:** the CLI option has changed (see Section 2.2)!

With the command line parameter “-P <LIBRARY-TYPE>” or “-P1 <LIBRARY-TYPE> -P2 <LIBRARY-TYPE>” the user can choose from the following supported library types:

RF-FIRSTSTRAND STRANDED library - first strand sequenced,

FR-SECONDSTRAND STRANDED library - second strand sequenced, and

UNSTRANDED UNSTRANDED library.

The “UNSTRANDED” library type is not available for *rt-arrest* and *lrt-arrest* method because an arrest site can not be unambiguously defined for this library type. Read tagging option “-B <BASE-SUB>” requires a stranded library type in order to correctly identify base substitutions based on strand information.

¹³Check: <http://www.htslib.org>

¹⁴Check: <https://broadinstitute.github.io/picard/explain-flags.html>

Table 2: Definition of an exemplary BED-like search region

contig	start	end
1	1000	1100
2	10000	10000

3.2 BED-like search region

Identification of interesting sites can be restricted to specific regions of the genome or transcriptome. Provide a minimalistic BED-like file to limit the search space to some region(s) or site(s). Complementary region(s) of the BAM files will not be considered, resulting in faster running time.

In the following file, the search is confined to a 100nt region on contig 1 starting at position 1,000 and a single site on contig 2 at position 10,000: Many individual sites may impair running performance of JACUSA2. Try to merge nearby sites to create contiguous regions and extract specific sites from JACUSA2 output with bedtools¹⁵ “intersect”:

Merge sites: `bedtools merge -d 500 singular_sites.bed > \`
`contiguous_regions.bed`

Run JACUSA2: `java -jar JACUSA2.jar call-2 \`
`-b contiguous_regions.bed -r JACUSA2.out <BAM files>`

Extract sites: `bedtools intersect -wa -a JACUSA2.out \`
`-b singular_sites.bed`

3.3 General output format

JACUSA2 writes its output to a user specified file. When using multiple threads, JACUSA2 will create a temporary file for each allocated thread in the temp directory that is provided by the JAVA Virtual Machine. Check the manual of your JAVA Virtual machine on how to change the temp directory.

Chosen command line parameters and current genomic position are printed to the command prompt and serve as a status guard. Furthermore, depending on the provided command line parameters, JACUSA2 will generate a file with sites that have been identified as potential artefacts when “-s [FILTERED-OUTPUT]” is provided.

Output format of JACUSA2 is controlled by the “-f <FORMAT>” command line option. Support for output formats depends on the used method. Check Table 3 for a summary of currently supported output formats.

Table 3: Summary of available output formats for each method.

Output format	Method			
	<i>call-1,2</i>	<i>pileup</i>	<i>rt-arrest</i>	<i>lrt-arrest</i>
JACUSA2 BED-like format	x	x	x	x
Variant Call Format (VCF ¹⁶)	x	x		

The default output format is a combination of BED6¹⁷ with JACUSA2 methods specific columns and common info columns: “info”, “filter”, and “ref”. The actual number of columns depends on the JACUSA2 method and the number of provided BAM files.

Check Table 4 and the following description general description of the JACUSA2.

(1, 2, 3) contig + start + end Name of the contig and 0-indexed coordinates [*start*, *end*).

(4) name String that depends on the method. Currently has no use except to ensure BED6 compatibility.

¹⁵<http://bedtools.readthedocs.org/en/latest/>

¹⁶Check: <http://samtools.github.io/hts-specs/VCFv4.1.pdf>

¹⁷Check: <http://genome.ucsc.edu/FAQ/FAQformat.html#format1>

Table 4: General description of JACUSA2 default output format with exemplary *call-2* output - “N” corresponds to the total number of columns. Check detailed explanation of columns under the table.

BED6 columns						Method specific			Additional		
1	2	3	4	5	6	7	...	N-3	N-2	N-1	N
1	100	101	call	8.07	-	0,0,0,10	...	0,2,0,10	*	* T	
		...					...			...	

- (5) **score** Score for this site. “*” if not available. Depends on actual method. Here, likelihood ratio that indicates how divergent base vectors between conditions are (higher value more divergent).
- (6) **strand** Possible values are: “.”, “+”, and “-” which correspond to “unstranded”, “positive strand”, and “negative strand”, respectively. If strand is not “.”, then columns with base call information will be showing base counts **according** to the strand - inverted base count if on the “negative strand”.
- (7 — N-3) **method specific** The number of base columns depends on the JACUSA2 method and the number of BAM files (check method specific explanation).
- (N-2) **info** The “info” field is used to add optional site specific data without changing the total number of columns. Actual content depends on method and command line parameters. The following optional data can be shown: Details about estimating the parameters of the underlying distribution. Insertion/deletion counts, and additional method specific data. If nothing provided, set to “*”.
- (N-1) **filter** Relevant, if artefact/feature filter *X* has been provided with “-a X” on the command line. The column will contain a “;”-separated list of artefact/feature filters that recognised this site. Here, the column could be “X”. Will be “*”, if empty.
- (N) **ref** The reference base according to coordinates (columns 1-3) and strand information (6) - Inverted, if on negative strand.

Check sections for method or option specific adjustments:

- *rt-arrest* method Section 5.3.1,
- *lrt-arrest* method Section 5.3.2,
- optional INDEL data fields Section 6.1, or
- base substitution based read stratification Section 6.3.

A “##” prefixed header that contains JACUSA2 runtime specific data such as version info and command line options is added to the default output format.

4 Artefact/feature filter

False positive variant calls can be related to mapping artefacts and sequencing technology. Short read mappers tend to produce incorrect alignments around INDEL positions that may be falsely identified as variant sites. Other false variant calls originate from uneven base call error distributions along short reads. We have implemented a panel of simple threshold based artefact filters. Beyond artefacts, we have added filters that identify some features of a pileup or a read and can be used to mark or exclude those sites.

When a site is identified by an artefact/feature filter, its corresponding ID *X* is added to the “filter” column and optionally this site can be moved to an other output file when command line “-s [FILTERED-OUTPUT]” has been used. This strategy preserves all sites and allows the user to investigate the fraction of filtered vs. total calls. In the following, we will use artefact and feature filtered interchangeably and we will call a site removed by a feature filter although the site has been marked by the ID of feature filter and can be easily identified in the “FILTERED-OUTPUT” file.

For example, when identifying RDDs by comparing gDNA vs. cDNA, it is common practice to remove or mask sites that are homozygous in gDNA (feature filter H) and have more than three distinct base types (artefact filter M).

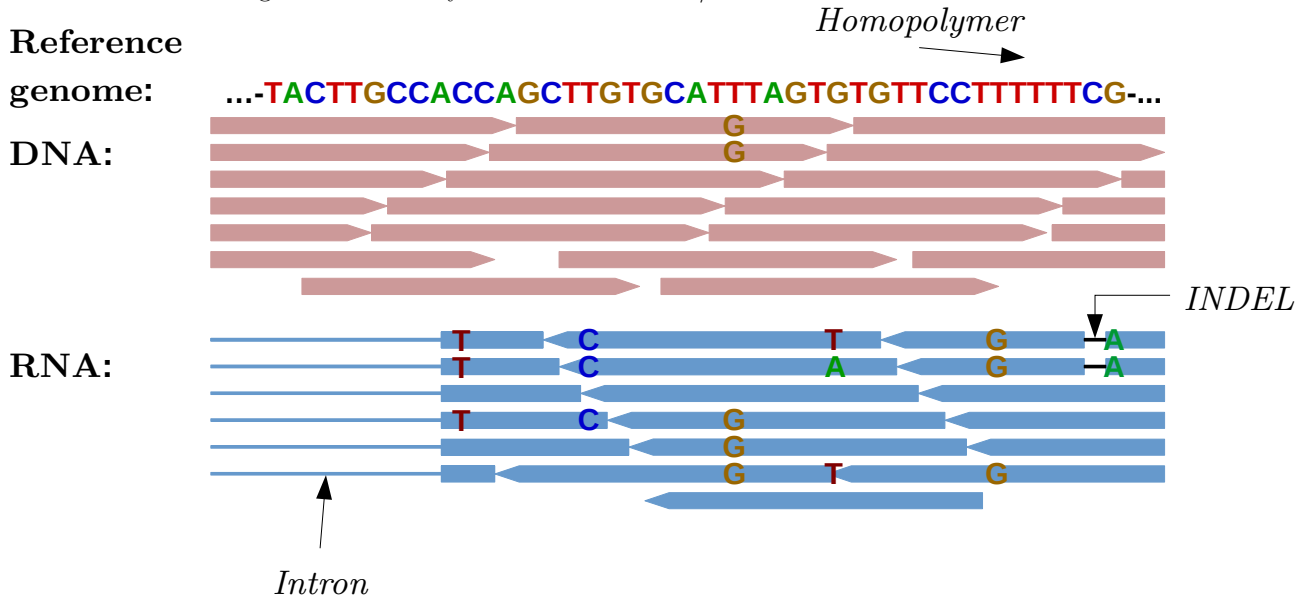
We have added a new exclude site filter (E) to JACUSA2 to mark sites in the final output that overlap with sites/regions that are stored in a file. The supported file type are: VCF, BED, or JACUSA2 output. Given a set of known SNPs, polymorphic sites can be directly marked and removed from the list of candidate RNA-editing sites:

```
java -jar JACUSA2.jar -a E:file=snp.vcf:type=VCF \
-r JACUSA2.out cDNA-1.bam cDNA-2.bam
```

For performance reasons, the input file is required to be sorted by coordinate — **WARNING:** Sort order is not tested within JACUSA2. The sort order of the “contig” column is not important.

Our filters (D,B,I,Y) monitor the distance d of a given candidate site to relevant read features such as start/end, INDEL positions, homopolymeric regions, and splice sites and remove the candidate site from further consideration if a proportion r of all reads falls below the given distance cutoff $\leq d$.

Figure 4: Summary of available artefact/feature filters for each method.



ID	Description	Artefact / feature Filter	JACUSA2 method		
			call-1	call-2, pileup	rt-arrest, lrt-arrest
B	Filter potential false positive variants adjacent to:		✓	✓	
I	- read start/end		✓	✓	✓
S	- INDEL position(s)		✓	✓	✓
D	- splice site(s)		✓	✓	✓
	Combines Filters:				
	- I + B + S		✓	✓	
	- I + S				✓
Y	Filter wrong variant calls within homopolymers		✓	✓	✓
M	Max allowed alleles per site		✓	✓	✓
H	Filter non-homozygous sites in condition 1 or 2		✓	✓	✓
E	Exclude sites contained in a file		✓	✓	✓

In JACUSA2, multi-line sites have been added to allow more complex data structures in the output. Currently, the “filter” column is only completely available for the summary of a site. For read/base stratified data, this will be the first line of a site. For *lrt-arrest* data this will be the line of a site where the arrest position column is “-”. The “filter” column of non-summary lines will be populated with the “?” value indicating unknown filter status. It is planned to change this behaviour in a future release to provide stratified filter information.

5 Usage

Calling JACUSA2 without any arguments will print the available tools which currently are:

```
java -jar JACUSA2.jar
METHOD      DESCRIPTION
call-1      Call variants - 1 condition
call-2      Call variants - 2 conditions
pileup      SAMtools like mpileup (2 conditions)
rt-arrest   Reverse Transcription Arrest - 2 conditions
lrt-arrest  Linkage arrest to base substitution - 2 conditions
[...]
```

5.1 Variant detection

In order to identify RNA editing sites by comparing gDNA and *stranded* RNA-Seq use:

first strand sequenced “-P1 UNSTRANDED -P2 RF-FIRSTSTRAND”

second strand sequenced “-P1 UNSTRANDED -P2 FR-SECONDSTRAND”

When your RNA-Seq is unstranded use: “-P UNSTRANDED” and infer the correct orientation from annotation. This sets the library type “UNSTRANDED” to all conditions.

Use the following command line to identify RNA-DNA differences in BAM files that might give rise to RNA editing sites:

```
java -jar JACUSA2.jar call-2 -r JACUSA2.out -s -a H:condition=1 \
  gDNA.bam cDNA.bam
```

Option “-a H:condition=c” ensures that polymorphic sites in gDNA will be marked as artefacts. The number $c \in \{1, 2\}$ defines which sample is required to be homozygous - in this case: “-a H:condition=1” will require gDNA.bam to be homozygous.

Use the following command line to identify RNA-RNA differences:

```
java -jar JACUSA2.jar call-2 -r JACUSA2.out -s cDNA_1.bam cDNA_2.bam
```

WARNING: If you want to identify RNA-RNA differences make sure NOT to use the filter “-a H[...]”! Otherwise, potential valid variants will be filtered out. If you happen to have a list of known polymorphic sites in a file “snps.{vcf|bed|JACUSA2 output}”, add the exclude site filter (E) to the previous statement:

```
java -jar JACUSA2.jar call-2 -a E:file=snps.vcf:type=VCF -r JACUSA2.out -s \
  cDNA_1.bam cDNA_2.bam
```

Polymorphic site defined in “snps.vcf” will be marked (see Section 7.1.3 for details.)

5.1.1 call-1

Single sample (call-1) allows to call variants against a reference. Internally, an *in silico* sample is created from information that is provided by the “MD” field in BAM files or by providing a <reference.fasta> via command line: “-R reference.fasta”.

The number of base columns depends on the number of BAM files. In basesIJ: *I* corresponds to sample and *J* to the respective replicate. Numbers indicate the base count of the following base vector: (A, C, G, T)

Sites that have a $>$ alleles are considered candidate variant sites and for this sites a test-score will be computed.

5.1.2 call-2

5.1.3 call-* output format

Column (5) is the test-statistic of the likelihood ratio test for comparing two conditions I and II . Base call count vectors D^I and D^{II} are modelled with the Dirichlet-Multinomial distribution (see [Piechotta et al., 2017] for details):

$$z = \log \frac{DirMult(\alpha^I; D^I) \cdot DirMult(\alpha^{II}; D^I I)}{DirMult(\alpha^{I,II}; D^I) \cdot DirMult(\alpha^{I,II}; D^{II})}$$

Parameters α^I , α^{II} , and $\alpha^{I,II}$ are estimated from base call vectors D^I , D^{II} , and $D^{I,II}$ (merged conditions) respectively, with a variant of the Newton iteration method presented by Minka¹⁸. Higher values of the test-statistic indicate a higher divergence of base call vectors between conditions.

When only one condition I is provided, *call-1* will create an *in silico* condition I^* by using available reference information to replace non-reference base calls in D^I and creating synthetic base call vectors D^{I^*} and applying the likelihood ratio test defined above. The output will contain only data for condition I .

Table 5: Exemplary *call-2* output on HEK-293 identifying RNA editing site by comparing DNA and RNA from [Piechotta et al., 2017]. The first²¹ and last²² columns have been removed for clarity. The first column is used to identify a line in the presented output.

1-4	5	6	7	8	9	10-13
#[...]	score	strand	bases11	bases12	bases21	[...]
...						...
...						...
...						...
:	:	:	:	:	:	:

5.2 pileup

See “Call variant - two samples” for details.

5.2.1 pileup output format

The output format of *pileup* is derieved from the output format of *call-**. The only modification is that the “score” column (5) corresponds to the total read coverage of a site (see Section 5.1.3 for further details).

5.3 *-arrest - Reverse transcriptase arrest events

JACUSA2 supports two methods to identify arrest events by means of comparing counts of arrest and through reads: *rt-arrest* and *lrt-arrest*. Beyond read counts, JACUSA2 shows base counts from arrest and through reads. This allows to inspect arrest events and variant calling simultaneously.

5.3.1 rt-arrest - 2 conditions

In this method, base call counts of arrest and read through reads are modelled by a Beta-Binomial distribution and differences between conditions are to be identified by means of a likelihood ratio test. Subsequent approximation with χ^2 distribution to compute a pvalue.

¹⁸Check: <https://tminka.github.io/papers/dirichlet/minka-dirichlet.pdf>

¹⁹First five columns: “contig”, “start”, “end”, and “name”

²⁰Last 3 columns: “info”, “filter”, and “ref”

²¹First five columns: “contig”, “start”, “end”, and “name”

²²Last 3 columns: “info”, “filter”, and “ref”

Sites are considered candidate arrest sites, if in all BAM files there is at least one read through AND one read arrest event. Otherwise, there would be no difference between the conditions. Furthermore, coverage filter and minBASQ of base call apply that will affect the output.

***rt-arrest* output format** In order to represent arrest events in JACUAS2 output, the base vector column “bases $_{ij}$ ” is split into two columns “arrest $_{ij}$ ” and “through $_{ij}$ ” (i specifies the condition and j the replicate. The former column represents base calls from reads that exhibit premature termination and the later correspond to base calls from reads without premature termination.

Table 6: Exemplary *rt-arrest* output of HIVRT +GMC vs. -GMC comparison from [Zhou et al., 2018]. The first²⁵ and last²⁶ columns have been removed to improve clarity clarity.

1-4	5	6	7	8	9-10	11	12-13
...	pvalue	...	arrest11	through11	...	info	...
...	0.0002		2875,154,15,956	26283,335,26,2276	...	arrest_score=14.15;	...
...	0.5347		0,0,0,0	1,0,209,0	...	arrest_score=0.39;	...
...	0.0281		601,715,150,1291	397,32422,144,314	...	arrest_score=4.82;	...
...	...	...	...	...	...	...	...

5.3.2 lrt-arrest - 2 conditions

lrt-arrest allows to link pileups to their arrest position. Output consists of read arrest and read through counts and a references to the associated arrest positions. There are cases, where currently an arrest position cannot be defined, e.g.: non properly paired reads.

***lrt-arrest* output format** The functionality and output format of *lrt-arrest* is currently under development. Be aware of future changes. Output consits of at least one line. Each separate arrest position adds an additional line. Any following site with identical coordinates (contig, start, end, strand) will have a different arrest position reference in the “arrest_pos” column.

This method supports partial artefact filtering. Currently, filters only apply to the unstratified data. Furthermore, coverage filter and minBASQ of Base Call apply that will affect the output.

6 Optional data

JACUSA2 data supports optional data that can be added by providing the appropriate command line option. See Table 7 for a summary of methods and supported optional data.

Table 7: Summary of available optional data for each method in JACUSA2. Command line options are provided within (“...”)

Optional data	Method		
	<i>call-*, pileup</i>	<i>rt-arrest</i>	<i>lrt-arrest</i>
INDELs:			
Insertions (“-I”)	✓	✓	
Deletions (“-D”)	✓	✓	
Read/base stratification (“-B <BASE-SUB>”)	✓	✓	

²³First five columns: “contig”, “start”, “end”, and “name”

²⁴Last 2 columns: “filter”, and “ref”

²⁵First five columns: “contig”, “start”, “end”, and “name”

²⁶Last 2 columns: “filter”, and “ref”

6.1 Adding INDEL statistics

Use “-I” or “-D” to add differential insertion or deletion data to the output. The “info” (N-2) column will have additional fields prefixed by “ins” or “del”. In the following, the description of deletions is presented. The underlying statistical test and output format for insertion and deletion data are identical.

6.2 Deletion output format

Add “-D” to your JACUSA2 run statement to add differential deletion scores for the HIVRT data from [Zhou et al., 2018] (see Section 2.3.2):

```
java -jar JACUSA2.jar rt-arrest -p 2 -P FR_SECONDSTRAND \
-D -r HIVRT.out HIVRT_+CMC.bam HIVRT_-CMC.bam
```

Example output for the “info” (N-2) column for contig NR_003286_RNA18SN5 and start 4 in the generated “HIVRT.out”:

```
[...]deletion_pvalue=0.098;deletion_score=2.734;deletions11=0,42109;deletions21=8,89123[...]
```

del_pvalue P-value from likelihood ratio test.

del_score Raw test-statistics of likelihoodratio test.

del_bases ij Condition(i) and replicate(j) specific counts of reads that contain a deletion (D) and total (T) reads at current location. E.g.: deletions21=31,236510; In condition 2 and replicate 1 there are 31 reads with a deletion of a total of 236.510 reads that span the current location. The total number of reads T includes reads that are overlapping from start to end with the reference including introns.

6.3 Adding read/base stratification

Read stratification or partitioning based on base substitution(s) can be enabled by adding “-B <BASE-SUB>” to your JACUSA2 run statement. <BASE-SUB> defines the base substitution $X2Y$: $X, Y \in \{A, C, G, T\}, X \neq Y$ of interest where X is the reference base and Y is a base call from some read. It is required to provide a stranded library type for each condition because otherwise $X2Y$ cannot unambiguously be determined from the sequencing data. It is possible to provide multiple base substitutions by separating each with a “,”.

For each site the output will consist of at least on line the represents the total not stratified reads. The “info” column will contain a field the following field “tag=*” indicating that the total reads are shown. If read with the wanted base substitution $A2G$ for example is encountered, all sites that are covered by this read will have an additional line of output and the “info” column will have a value of “tag=A2G”.

7 Description of command line options

7.1 Input / Output

7.1.1 Input BAM files

7.1.2 Output file

-r RESULT-FILE

results are written to RESULT-FILE

call-1
call-2
pileup
rt-arrest
lrt-arrest

7.1.3 Output artefacts to separate file

-s FILTERED-FILE	Store feature-filtered results in another file (= RESULT-FILE.filtered if no argument) or (= FILTERED-FILE)	call-1 call-2 pileup rt-arrest lrt-arrest
------------------	---	---

7.1.4 Show all sites

-A	Show all sites - including sites without variants	call-1 call-2
----	---	------------------

7.2 Input BED file

-b BED	BED file to scan for variants	call-1 call-2 pileup rt-arrest lrt-arrest
--------	-------------------------------	---

7.3 Reference fasta file

-R REF-FASTA	use reference FASTA file (must be indexed)	call-1 call-2 pileup rt-arrest lrt-arrest
--------------	--	---

7.4 Library type

-P LIB-TYPE	Choose the library type for all conditions: RF-FIRSTSTRAND library - first strand sequenced FR-SECONDSTRAND library - second strand sequenced UNSTRANDED library default: UNSTRANDED lrt-arrest	call-1 call-2 pileup rt-arrest
-P1 LIB-TYPE	Choose the library type for condition 1: RF-FIRSTSTRAND library - first strand sequenced FR-SECONDSTRAND library - second strand sequenced UNSTRANDED library default: UNSTRANDED	call-2 pileup rt-arrest lrt-arrest
-P2 LIB-TYPE	Choose the library type for condition 2: RF-FIRSTSTRAND library - first strand sequenced FR-SECONDSTRAND library - second strand sequenced UNSTRANDED library default: UNSTRANDED	call-2 pileup rt-arrest lrt-arrest

7.5 Read/base stratification

-B READ-TAG	Tag reads by base substitution. Count non-reference base substitution per read and stratify. Requires stranded library type. (Format for T to C mismatch: T2C; use ',' to separate substitutions) Default: none	call-1 call-2 pileup rt-arrest
-------------	---	---

7.6 Show INDEL statistics

7.6.1 Show insertion statistics

-I	Show insertion score	call-1 call-2 pileup rt-arrest
----	----------------------	---

7.6.2 Show deletion statistics

-D	Show deletion score	call-1 call-2 pileup rt-arrest
----	---------------------	---

7.7 General filtering

7.7.1 Filter by mapping quality

	filter positions with MAPQ < MIN-MAPQ default: -1	call-1 call-2
-m MIN-MAPQ	filter positions with MAPQ < MIN-MAPQ for all conditions default: 20	pileup rt-arrest lrt-arrest
-m1 MIN-MAPQ1	filter positions with MAPQ < MIN-MAPQ1 for condition 1 default: 20	call-2 pileup rt-arrest lrt-arrest
-m2 MIN-MAPQ2	filter positions with MAPQ < MIN-MAPQ2 for condition 2 default: 20	call-2 pileup rt-arrest lrt-arrest

7.7.2 Filter by base call quality

	filter positions with base quality < MIN-BASQ default: 20	call-1 call-2
-q MIN-BASQ	filter positions with base quality < MIN-BASQ for all conditions default: 20	pileup rt-arrest lrt-arrest
-q1 MIN-BASQ1	filter positions with base quality < MIN-BASQ1 for condition 1 default: 20	call-2 pileup rt-arrest lrt-arrest
-q2 MIN-BASQ2	filter positions with base quality < MIN-BASQ2 for condition 2 default: 20	call-2 pileup rt-arrest lrt-arrest

7.7.3 Enforce/override base call quality

	enforce base quality score BASQ for all base calls; default: use existing BASQ	call-1
-Q BASQ	enforce base quality score BASQ for all base calls for all conditions; default: use existing BASQ	call-2 pileup rt-arrest lrt-arrest

7.7.4 Filter by minimal coverage

	filter positions with coverage < MIN-COVERAGE default: 5	call-1 call-2
-c MIN-COVERAGE	filter positions with coverage < MIN-COVERAGE for all conditions default: 5	pileup rt-arrest lrt-arrest
-c1 MIN-COVERAGE1	filter positions with coverage < MIN-COVERAGE1 for condition 1 default: 5	call-2 pileup rt-arrest lrt-arrest
-c2 MIN-COVERAGE2	filter positions with coverage < MIN-COVERAGE2 for condition 2 default: 5	call-2 pileup rt-arrest lrt-arrest

7.7.5 Limit maximal depth

-d1 MAX-DEPTH1	max read depth for condition 1 default: -1	call-2 pileup lrt-arrest
----------------	--	--------------------------------

7.7.6 Filter by flag(s)

-f OUTPUT-FORMAT	Choose output format: <*> B: BED6-extended result format < > V: VCF Output format. Option -P will be ignored (VCF is unstranded)	call-1
	Choose output format: <*> B: BED6-extended result format < > V: VCF Output format. Option -P will be ignored (VCF is unstranded)	call-2
	Choose output format: <*> B: Default < > M: samtools mpileup like format	pileup

7.7.7 Retain by flag(s)

	filter reads with flags FLAG default: 0	call-1 call-2
-F FLAG	filter reads with flags FLAG for all conditions default: 0	pileup rt-arrest lrt-arrest
-F1 FLAG1	filter reads with flags FLAG1 for condition 1 default: 0	call-2 pileup rt-arrest lrt-arrest
-F2 FLAG2	filter reads with flags FLAG2 for condition 2 default: 0	call-2 pileup rt-arrest lrt-arrest

7.7.8 Filter by number of hits

-filterNH NH	Max NH-VALUE for SAM tag NH for all conditions	call-1 call-2 pileup rt-arrest lrt-arrest
-filterNH_1 NH	Max NH-VALUE for SAM tag NH for condition 1	call-2 pileup rt-arrest lrt-arrest
-filterNH_2 NH	Max NH-VALUE for SAM tag NH for condition 2	call-2 pileup rt-arrest lrt-arrest

7.7.9 Filter by number of mismatches

-filterNM NM	Max NM-VALUE for SAM tag NM for all conditions	call-1 call-2 pileup rt-arrest lrt-arrest
-filterNM_1 NM	Max NM-VALUE for SAM tag NM for condition 1	call-2 pileup rt-arrest lrt-arrest
-filterNM_2 NM	Max NM-VALUE for SAM tag NM for condition 2	call-2 pileup rt-arrest lrt-arrest

7.8 Artefact/feature filter

-a FEATURE-FILTER	[...] Use -h to see extended help	call-1 call-2 pileup rt-arrest lrt-arrest
-------------------	-----------------------------------	---

7.9 Thread related

7.9.1 Number of parallel threads

-p THREADS	use # THREADS default: 1	call-1 call-2 pileup rt-arrest lrt-arrest
------------	--------------------------	---

7.9.2 Actual thread window size

-w WINDOW	size of the window used for caching. Make sure this is greater than the read size default: 10000	call-1 call-2 pileup
	size of the window used for caching. Make sure this is greater than the read size default: 10000	rt-arrest
	size of the window used for caching. Make sure this is greater than the read size default: 5000	lrt-arrest

7.9.3 Reserved thread window size

-W THREAD-WINDOW	size of the window used per thread default: 100000	call-1
		call-2
		pileup
		rt-arrest lrt-arrest

7.10 Test-statistic options

-T THRESHOLD	Filter positions based on test-statistic THRESHOLD default: DO NOT FILTER	call-1
		call-2
		rt-arrest
		lrt-arrest
-u MODE	[...] Use -h to see extended help	call-1
		call-2
		rt-arrest
		lrt-arrest

7.11 Misc

-h	Print extended usage information	call-1
		call-2
		pileup
		rt-arrest lrt-arrest
-x	turn on Debug modus	call-1
		call-2
		pileup
		rt-arrest lrt-arrest

8 Used libraries

Library	Source
htsjdk 4.1.3	https://github.com/samtools/htsjdk
Apache:	
commons-cli 1.4	https://commons.apache.org/proper/commons-cli
commons-math3 3.6.1	https://commons.apache.org/proper/commons-math

References

- [Li et al., 2009] Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., and and, R. D. (2009). The sequence alignment/map format and SAMtools. *Bioinformatics*, 25(16):2078–2079.
- [Piechotta et al., 2017] Piechotta, M., Wyler, E., Ohler, U., Landthaler, M., and Dieterich, C. (2017). JACUSA: site-specific identification of RNA editing events from replicate sequencing data. *BMC Bioinformatics*, 18(1).

- [Piekna-Przybylska et al., 2007] Piekna-Przybylska, D., Decatur, W. A., and Fournier, M. J. (2007). The 3d rRNA modification maps database: with interactive tools for ribosome analysis. *Nucleic Acids Research*, 36(Database):D178–D183.
- [Zhou et al., 2018] Zhou, K. I., Clark, W. C., Pan, D. W., Eckwahl, M. J., Dai, Q., and Pan, T. (2018). Pseudouridines have context-dependent mutation and stop rates in high-throughput sequencing. *RNA Biology*, 15(7):892–900.